

Firestore & Firebase Interview Master Notes

SDE / Web Development Prep

October 5, 2026

1 Module 1: Architecture & Core Fundamentals

1.1 Theory

- **Backend-as-a-Service (BaaS):** Firebase removes the overhead of managing infrastructure, provisioning database clusters, and writing custom boilerplate authentication layers. It shifts direct communication to client SDKs while relying on managed cloud infrastructure.
- **Client SDK vs. Admin SDK:**
 - *Client SDK:* Runs in untrusted environments (browsers, mobile apps). It is strictly bound by **Firestore Security Rules** to protect database integrity.
 - *Admin SDK:* Runs exclusively in trusted server environments (Node.js, Cloud Functions). It authenticates via a Service Account and **bypasses all Security Rules**, giving it root-level access to perform administrative operations.

1.2 Code Snippets

Listing 1: Client SDK Initialization (Web)

```
import { initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";
import { getFirestore } from "firebase/firestore";

const firebaseConfig = {
  apiKey: "AIzaSy...",
  authDomain: "your-app.firebaseio.com",
  projectId: "your-app-id"
};

const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const db = getFirestore(app);
```

Listing 2: Admin SDK Initialization (Node.js Backend)

```
const admin = require("firebase-admin");
const serviceAccount = require("../serviceAccountKey.json");

admin.initializeApp({
  credential: admin.credential.cert(serviceAccount)
});

const adminDb = admin.firestore();
const adminAuth = admin.auth();
```

2 Module 2: Firebase Authentication & Identity Management

2.1 Theory

- **Token Architecture:** Firebase uses short-lived **JSON Web Tokens (JWTs)** known as Firebase ID tokens. These are verified automatically by Firestore and Cloud Functions. Long-lived refresh tokens manage session continuity behind the scenes.
- **Role-Based Access Control (RBAC):** To implement custom roles (e.g., `admin`, `moderator`), developers use the Admin SDK to assign **Custom Claims** to a user's token.

2.2 Code Snippets

Listing 3: Assigning Custom Claims using Admin SDK (Backend)

```
async function setAdminRole(uid) {
  await admin.auth().setCustomUserClaims(uid, { admin: true });
  console.log('Custom claims set for user: ${uid}');
}
```

Listing 4: Handling Auth State Changes (Client-side Frontend)

```
import { onAuthStateChanged } from "firebase/auth";

onAuthStateChanged(auth, (user) => {
  if (user) {
    console.log("User is logged in:", user.uid);
    user.getIdTokenResult().then((tokenResult) => {
      const isAdmin = tokenResult.claims.admin || false;
      console.log("Is Admin:", isAdmin);
    });
  } else {
    console.log("User is logged out");
  }
});
```

3 Module 3: Cloud Firestore Data Modeling & Querying

3.1 Theory

- **NoSQL Document Paradigm:** Firestore stores data inside **Documents** (key-value maps) organized into **Collections** and nested **Subcollections**.
- **Denormalization:** Because Firestore does not support server-side SQL-style joins, data is intentionally duplicated across documents to optimize read performance.
- **Indexing:** Simple queries use single-field auto-indexes. Complex queries involving multiple fields or inequalities require **Composite Indexes**.

3.2 Code Snippets

Listing 5: Performing a Query with Filtering, Ordering, and Pagination

```
import { collection, query, where, orderBy, limit, startAfter, getDocs } from "
  firebase/firestore";

async function fetchPaginatedPosts(lastVisibleDoc = null) {
  let q = query(
    collection(db, "posts"),
    where("status", "==", "published"),
    orderBy("createdAt", "desc"),
    limit(10),
    startAfter(lastVisibleDoc)
  );
  const snapshot = await getDocs(q);
  return snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }));
}
```

```

    limit(10)
  );

  if (lastVisibleDoc) {
    q = query(q, startAfter(lastVisibleDoc));
  }

  const querySnapshot = await getDocs(q);
  const posts = querySnapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }));
  const lastDoc = querySnapshot.docs[querySnapshot.docs.length - 1];

  return { posts, lastDoc };
}

```

4 Module 4: Firestore Operations, Transactions & Offline Sync

4.1 Theory

- **Batched Writes vs. Transactions:**

- *Batched Writes:* Execute multiple independent write operations atomically.
- *Transactions:* Execute read-then-write operations atomically using optimistic concurrency control (retrying automatically if underlying data changes).

- **Offline Persistence:** The SDK caches data locally using IndexedDB/SQLite, queuing writes when offline and executing a **Last-Write-Wins (LWW)** sync strategy upon reconnection.

4.2 Code Snippets

Listing 6: Using a Firestore Transaction for Atomic Counter / Inventory Check

```

import { runTransaction, doc } from "firebase/firestore";

async function purchaseItem(itemId) {
  const itemRef = doc(db, "items", itemId);

  try {
    await runTransaction(db, async (transaction) => {
      const itemDoc = await transaction.get(itemRef);
      if (!itemDoc.exists()) {
        throw "Item does not exist!";
      }

      const newStock = itemDoc.data().stock - 1;
      if (newStock < 0) {
        throw "Item is out of stock!";
      }

      transaction.update(itemRef, { stock: newStock });
    });
    console.log("Transaction successfully committed!");
  } catch (e) {
    console.error("Transaction failed: ", e);
  }
}

```

5 Module 5: Firestore Security Rules

5.1 Theory

- **Contextual Objects:** Rules leverage `request.auth` (user context), `request.resource.data` (incoming payload), and `resource.data` (existing state) to enforce access policies.
- **Data Validation:** Rules serve as a final schema validation barrier to ensure type safety and prevent unauthorized mutations.

5.2 Code Snippets

Listing 7: Firestore Security Rules Example

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // User profile: Only owner can read and write their document
    match /users/{userId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }

    // Posts: Public read, authenticated create with validation
    match /posts/{postId} {
      allow read: if true;
      allow create: if request.auth != null
                     && request.resource.data.title is string
                     && request.resource.data.title.size() > 0;
    }
  }
}
```

6 Module 6: Cloud Functions & Asynchronous Workflows

6.1 Theory

- **Trigger Categories:** Functions can be exposed via HTTP endpoints, client-side **Callable Functions** (which pass authentication context automatically), or background event hooks (`onCreate`, `onUpdate`).
- **Idempotency:** Because background triggers can fire multiple times under network partitions, server functions must be designed to handle duplicate events safely.

6.2 Code Snippets

Listing 8: Cloud Function Background Trigger (Firestore onCreate)

```
const { onDocumentCreated } = require("firebase-functions/v2/firestore");
const admin = require("firebase-admin");

exports.sendWelcomeEmail = onDocumentCreated("users/{userId}", async (event) => {
  {
    const snapshot = event.data;
    if (!snapshot) return;

    const userData = snapshot.data();
    const email = userData.email;

    console.log(`Sending welcome email to: ${email}`);
    return null;
  }
});
```

7 Module 7: Optimization, Scaling & System Design

7.1 Theory

- **Cost Optimization:** Reduce read operations by leveraging client-side snapshot listeners, local caching, and native aggregation queries (`getCountFromServer()`).
- **High-Throughput Scaling:** Overcome the 1-write-per-second document limitation by implementing **Distributed Sharded Counters** for high-frequency updates like post likes or view counts.